

FDU 并行计算 3. 并行编程

本文参考以下教材:

- 并行计算: 结构·算法·编程 (第 3 版, 陈国良) 第 13, 14, 15, 16 章

欢迎批评指正!

3.1 基础知识

3.2 共享存储编程

3.2.1 基于共享变量

共享存储并行编程以**共享变量**作为进程 (或线程) 之间的主要通信方式, 各处理器可直接读写共享存储器中的数据, 无需显式的数据传送. 尽管分布式系统在峰值性能上占优, 但共享存储系统编程更简单.

共享存储并程序序设计面临的首要问题是**任务划分**.

单程序多数据流 (SPMD) 通常采用域分解的方法, 将计算区域划分为若干子域以实现数据并行; 而多程序多数据流 (MPMD) 则通过功能分解, 将不同子任务分配给不同处理器.

任务调度是共享存储并行编程中的另一关键问题.

静态调度在程序设计阶段或编译阶段确定任务分配, 动态调度则在运行时根据系统状态动态完成分配.

任务同步在共享存储并程序序中用于协调任务的执行次序以及对共享变量的安全访问.

常见的同步方式包括锁、路障、事件、信号灯和管程等, 不恰当的同步设计可能导致死锁等严重问题.

从系统环境角度看, 共享存储并行编程可分为纯共享存储环境和虚拟共享存储环境.

纯共享存储环境对应于 SMP 架构, 采用统一内存访问 (UMA) 模型, 系统提供集中且全局统一编址的共享存储, 典型的现代编程标准包括 Pthreads 和 OpenMP.

虚拟共享存储环境则建立在分布式存储之上, 通过操作系统提供共享虚拟存储空间的抽象, 其访存模型通常为 NUMA, 代表性的编程系统包括 Linda 和 Jiajia.

3.2.2 线程编程

(Pthreads 的相关知识详见 FDU 操作系统 3. 并发.pdf)

QtConcurrent / QtThreads 的高层次编程接口旨在让程序员以任务级并行的方式利用多核资源, 而不必直接管理线程的创建、同步与销毁.

在高层抽象中, Qt 将并行计算视为对**容器数据的变换过程**.

典型的例子是 `filter`、`map` 和 `reduce` 等操作.

以 `filter` 为例, 其语义是对一个输入序列中的每个元素并行地应用谓词函数, 并保留满足条件的元素.

QtConcurrent 会自动将容器划分为若干子区间, 调度到内部维护的线程池中执行, 用户只需提供一个无副作用的判断函数即可.

Filter-Reduce 示例 (筛选偶数, 平方求和):

```
#include <QtConcurrent>
#include <QVector>
#include <QDebug>

/*
 * filter 函数
 * 要求无副作用 (不访问共享可变状态) 且可被并行安全调用
 */
bool isEven(int x)
{
    return (x % 2) == 0;
}

/*
 * reduce 函数
 * QtConcurrent 会在并行阶段为每个线程维护局部 result, 最终再将它们合并
 */
void sumOfSquares(int &result, int value)
{
    result += value * value;
}

int main()
{
    // 输入数据
    QVector<int> data;
    for (int i = 1; i <= 100; ++i)
        data.append(i);

    /* 启动 filter-reduce 计算 */
    int result = QtConcurrent::filteredReduced(
        data,           // 输入数据
        isEven,         // filter: 筛选偶数
        sumOfSquares,   // reduce: 平方求和
        QtConcurrent::UnorderedReduce
    );

    qDebug() << "Sum of squares of even numbers =" << result;
    return 0;
}
```

Map-Reduce 示例 (并发归并排序):

```
#include <QtConcurrent>
#include <QVector>
#include <QDebug>
#include <algorithm>
```

```

/*
 * map 函数
 * 输入： 一个整数
 * 输出： 只包含该整数的有序数组
 */
using Chunk = QVector<int>;
Chunk mapToSortedChunk(int value)
{
    Chunk chunk;
    chunk.append(value);
    return chunk;
}

/*
 * reduce 函数： 将两个已排序数组归并成一个新的有序数组
 * result : 累积的有序数组
 * chunk   : 新产生的有序子数组
 */
void mergeChunks(Chunk &result, const Chunk &chunk)
{
    // 若 result 为空，直接接管
    if (result.isEmpty()) {
        result = chunk;
        return;
    }

    Chunk merged;
    merged.reserve(result.size() + chunk.size());

    int i = 0, j = 0;

    // 标准归并过程
    while (i < result.size() && j < chunk.size()) {
        if (result[i] <= chunk[j])
            merged.append(result[i++]);
        else
            merged.append(chunk[j++]);
    }

    while (i < result.size())
        merged.append(result[i++]);

    while (j < chunk.size())
        merged.append(chunk[j++]);

    result.swap(merged);
}

int main()
{
    QVector<int> data = {9, 3, 7, 1, 6, 2, 8, 5, 4};

    /* 启动 map-reduce 计算 */

```

```

    chunk sorted = QtConcurrent::mappedReduced(
        data,                      // 输入数据
        mapToSortedChunk,         // map: 并行产生局部有序段
        mergeChunks,              // reduce: 逐步归并这些有序段
        QtConcurrent::UnorderedReduce
    );

    qDebug() << "Sorted result:" << sorted;
    return 0;
}

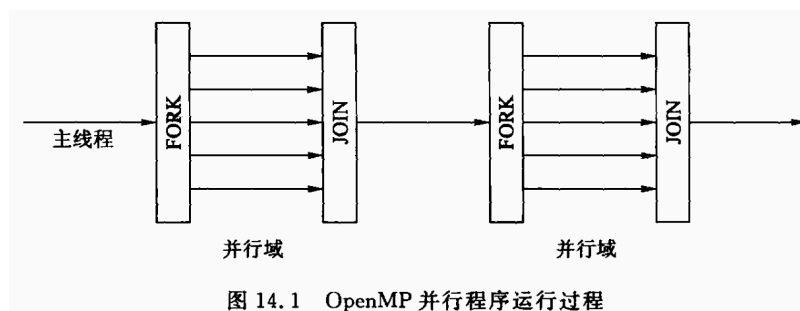
```

3.2.3 OpenMP 编程

OpenMP 是一种面向**共享存储体系结构**的并行编程模型。

它通过 `#pragma` 预处理指令、运行时库函数和环境变量三者的组合，将串行程序的增量编程转化为并行，封装了线程管理、同步与调度等复杂实现细节，但相应地也降低了程序员对并行执行细节的精细控制能力。

由于 OpenMP 采用 **fork-join** 执行模型，程序在进入并行区域时由主线程派生出一组工作线程并行执行，退出并行区域后再汇合为单线程继续运行，因此其并行结构清晰，易于理解和调试。



- **共享变量:** 所有声明在并行域外部的变量默认都是共享变量 (除循环索引变量以外)
- **私有变量:** 所有声明在并行域内部的变量都被分配在每个线程的运行栈上，在并行域结束时被销毁
- **规约变量:** 一个特殊类型的共享变量，并行域的每个线程都有该变量的一个副本。
并行域结束时，会有一个操作 (如求和操作) 作用在每个副本上，结果保存在共享变量中。

并行梯形数值积分示例 (计算 π):

```

#include <stdio.h>
#include <omp.h>

/*
 * 被积函数  $f(x) = 4 / (1 + x^2)$ ，其在  $[0,1]$  上的积分等于  $\pi$ 
 */
double f(double x)
{
    return 4.0 / (1.0 + x * x);
}

int main()
{
    int n = 100000000;          // 将区间  $[0,1]$  均分为 n 个小区间

```

```

double a = 0.0, b = 1.0;
double h = (b - a) / n; // 每个小区间的宽度
double sum = 0.0;        // 用于累加积分值

/*
 * openMP 并行区域
 * - parallel for: 并行化 for 循环
 * - reduction(+:sum): 对 sum 执行并行归约, 避免数据竞争
 */
#pragma omp parallel for reduction(+:sum)
for (int i = 0; i < n; ++i)
{
    double x_left = a + i * h;
    double x_right = x_left + h;
    sum += 0.5 * (f(x_left) + f(x_right)) * h;
}

printf("Approximation of pi = %.15f\n", sum);
return 0;
}

```

3.3 CUDA 编程

3.3.1 基础语法

CUDA 采用**主机 (CPU)** $\leftrightarrow$ **设备 (GPU)** 的异构编程模型。

普通 C/C++ 代码在主机上执行, 而标注为核函数的代码在设备上由大量线程并行执行。

两者的函数语法首先通过函数限定符区分:

`__global__` 修饰的函数称为核函数, 只能从主机端调用、在设备端执行;

`__device__` 修饰的函数只能在设备端调用并执行;

`__host__` 表示普通的主机函数, 若省略则默认为 `__host__`,

`__host__ __device__` 组合表示该函数既可在主机端也可在设备端调用。

核函数的调用语法采用三尖括号的执行配置形式: `kernel<<<gridDim, blockDim>>>(args...)`,

其中 `gridDim` 指定线程网格中 block 的数量, `blockDim` 指定每个 block 中线程的数。

在核函数内部, 每个线程都可以通过内建变量获知自身的唯一标识。

`threadIdx` 表示线程在当前 block 内的索引, `blockIdx` 表示 block 在 grid 中的索引,

`blockDim` 表示每个 block 的线程维度, `gridDim` 表示整个 grid 的 block 维度。

这些变量通常是 `dim3` 类型, 用于支持一维、二维或三维线程组织。

最常见的一维全局索引计算形式为 `int idx = blockIdx.x * blockDim.x + threadIdx.x;`

它将层次化的线程编号映射为线性数据索引。

CUDA 中的内存分为主机内存和设备内存, 二者物理上分离, 因此需要显式管理数据传输。

设备内存通常通过 `cudaMalloc` 分配, 通过 `cudaFree` 释放;

主机与设备之间的数据拷贝使用 `cudaMemcpy`,

并通过 `cudaMemcpyHostToDevice`、`cudaMemcpyDeviceToHost` 等枚举值指定拷贝方向。

在核函数内部，CUDA 提供多级存储空间。

最基本的是 `global` 内存，对所有线程可见但访问延迟较高；

`shared` 内存存在同一 `block` 内线程共享，具有低延迟，常用于数据复用与协作计算；

`local` 内存是每个线程私有的，通常映射到寄存器或本地内存。

`__syncthreads()` 用于 `block` 内线程的路障同步，保证所有线程在继续执行前都到达该点。

需要强调的是，CUDA 不提供跨 `block` 的同步原语，

因此 `block` 之间在一次 `kernel` 启动中是相互独立的，这一限制直接影响算法设计方式。

3.3.2 简单示例

```
#include <stdio.h>
#include <cuda_runtime.h>

// 设备函数，仅能在设备上调用
__device__ float add(float x, float y) {

    return x + y;
}

// 核函数：矩阵加法
__global__ void matrixAdd(const float* A, const float* B, float* C, int width, int height) {
    // 线程在 block 内的二维索引
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // block 在 grid 中的二维索引
    int bx = blockIdx.x;
    int by = blockIdx.y;

    // block 大小
    int bdx = blockDim.x;
    int bdy = blockDim.y;

    // 全局线程索引（映射到矩阵行列）
    int col = bx * bdx + tx;
    int row = by * bdy + ty;

    // 边界检查
    if (col < width && row < height) {
        int idx = row * width + col; // 一维线性索引

        // 调用设备函数
        C[idx] = add(A[idx], B[idx]);

        // 线程同步（不是必需的）
        // __syncthreads();
    }
}
```

```

int main() {
    // 主机 (CPU) 变量
    const int width = 1024;
    const int height = 1024;
    size_t size = width * height * sizeof(float);

    float *h_A = new float[width * height];
    float *h_B = new float[width * height];
    float *h_C = new float[width * height];

    // 初始化矩阵
    for (int i = 0; i < width * height; ++i) {
        h_A[i] = i * 0.001f;
        h_B[i] = i * 0.002f;
    }

    // 设备 (GPU) 内存分配
    float *d_A, *d_B, *d_C;
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);
    cudaMalloc((void**)&d_C, size);

    // 数据从主机拷贝到设备
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // 配置线程网格
    dim3 threadsPerBlock(16, 16); // 每个 block 16x16 线程
    dim3 numBlocks((width + 15)/16, (height + 15)/16); // 计算 grid 大小, 向上取整

    // 调用核函数
    matrixAdd<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, width, height);

    // 等待 GPU 完成
    cudaDeviceSynchronize();

    // 结果从设备拷贝回主机
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // 简单验证
    printf("C[0] = %f, C[width*height-1] = %f\n", h_C[0], h_C[width*height-1]);

    // 释放内存
    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    return 0;
}

```

3.4 分布存储编程

3.4.1 基于消息传递

基于消息传递的并行编程要求用户显式地通过发送和接收消息来实现处理器间的数据交换，每个进程拥有独立的地址空间，不能直接访问其他进程的数据，这种方式适合大粒度并行。

在 SPMD (单程序多数据流) 模式下，问题被划分为多个子区域 (域分解)，每个处理器运行同一程序但处理不同数据。

SPMD 程序可以采用主机/节点结构或无主机结构：

主机/节点结构中，控制节点负责程序管理、I/O 和收集结果，而计算节点执行局部计算；

无主机结构则所有计算节点均运行相同程序，控制和计算均在节点内部完成。

无主机结构更易开发、调试和移植。

在 MPMD (多程序多数据流) 模式下，每个处理器执行不同的程序副本 (函数分解)，各自完成不同计算。

MPMD 程序的实现可采用数据流方式或客户-服务器方式：

数据流方式按子问题间的数据依赖顺序驱动计算，适用于异构环境；

客户-服务器方式将通用子功能部署为服务器，客户端根据需求请求服务，适用于资源共享或通用服务场景。

这种模式的特点是处理器角色不对等，服务器提供资源或计算能力，客户端发起请求，能有效组织异构计算资源并重用通用功能。

3.4.2 MPI 编程

MPI 是基于消息传递的并行编程标准，它提供了跨进程的数据交换机制，每个进程都有自己独立的地址空间。

MPI 程序通常以串行程序形式编写，但在运行时，每个进程在不同计算节点上执行相同或不同的程序副本。

MPI 的基本语法通过一组函数调用实现，几乎所有接口都以 `MPI_` 前缀命名。

任何 MPI 程序都从初始化开始，

调用 `MPI_Init(&argc, &argv)` 来启动 MPI 环境，使得进程间的通信机制可用。

紧接着，程序通常需要获取进程总数和当前进程编号，

分别使用 `MPI_Comm_size(MPI_COMM_WORLD, &size)` 和 `MPI_Comm_rank(MPI_COMM_WORLD, &rank)`。

`MPI_COMM_WORLD` 是默认的通信子，包含所有启动的进程；

`size` 表示总进程数，`rank` 表示进程编号，编号从 0 开始。

进程间通信分为点对点通信和集合通信。

点对点通信主要使用 `MPI_Send` 发送消息和 `MPI_Recv` 接收消息，

其中需要指定消息的起始地址、元素个数、数据类型、目标/源进程编号以及通信标签。

MPI 提供的数据类型如 `MPI_INT`、`MPI_FLOAT`、`MPI_DOUBLE` 等，用于描述消息中的基本数据类型。

MPI 的发送和接收操作可以是阻塞或非阻塞，阻塞操作会等待通信完成，

非阻塞操作 (如 `MPI_Isend`、`MPI_Irecv`) 允许进程在等待通信完成的同时继续执行其他计算。

集合通信用于所有进程间的协调操作，例如广播 `MPI_Bcast`、归约 `MPI_Reduce`、

全局归约 `MPI_Allreduce`、散播 `MPI_Scatter` 和收集 `MPI_Gather` 等。

集合通信通常自动处理同步和数据分发，适合全局操作和结果聚合。

MPI 还提供同步和终止机制.

`MPI_Barrier` 用于全体进程的同步, 保证所有进程在进入下一阶段前都达到该屏障.

程序执行完成后, 必须调用 `MPI_Finalize()` 释放 MPI 环境资源, 使进程可以正常退出.

简单演示:

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int rank, size;

    // 1. 初始化 MPI 环境
    MPI_Init(&argc, &argv);

    // 2. 获取总进程数和当前进程编号
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // 每个进程生成一个随机数
    srand(rank + 1); // 不同进程不同种子
    int local_value = rand() % 100;
    printf("Process %d/%d has local_value = %d\n", rank, size, local_value);

    // 3. 广播操作: 让进程 0 的值被所有进程共享
    int broadcast_value;
    if (rank == 0) {
        broadcast_value = local_value; // 进程 0 提供广播值
    }
    MPI_Bcast(&broadcast_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received broadcast_value = %d\n", rank, broadcast_value);

    // 4. 点对点通信示例: 进程 i 向进程 (i+1)%size 发送自己的 local_value
    int recv_value = -1;
    int dest = (rank + 1) % size;
    int source = (rank - 1 + size) % size;

    MPI_Send(&local_value, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
    MPI_Recv(&recv_value, 1, MPI_INT, source, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received recv_value = %d from process %d\n", rank, recv_value,
    source);

    // 5. 全局归约: 计算所有进程 local_value 的总和
    int sum;
    MPI_Reduce(&local_value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Sum of all local_values = %d\n", sum);
    }

    // 6. 全体进程同步
    MPI_Barrier(MPI_COMM_WORLD);
    if (rank == 0) {
```

```

        printf("All processes reached the barrier.\n");
    }

    // 7. 释放 MPI 环境
    MPI_Finalize();
    return 0;
}

```

近似计算 π 的简单示例:

```

#include <mpi.h>
#include <stdio.h>

#define N 100000 // 积分区间划分的步数

int main(int argc, char** argv) {
    int rank, size; // 进程编号和总进程数
    double w = 1.0 / N; // 每个小矩形的宽度
    double local = 0.0; // 当前进程累加的局部和
    double pi = 0.0; // 最终结果
    double x;

    // 1. 初始化 MPI 环境
    MPI_Init(&argc, &argv);

    // 2. 获取当前进程编号和总进程数
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // 3. 每个进程计算属于自己的区间
    // 使用步长为 size 的循环实现负载均衡
    for (int i = rank; i < N; i += size) {
        x = (i + 0.5) * w;
        local += 4.0 / (1.0 + x * x); // 局部累加
    }

    // 4. 全局归约: 将各进程的局部和累加到 pi
    MPI_Reduce(&local, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    // 5. 仅由进程 0 输出最终结果
    if (rank == 0) {
        pi *= w; // 乘以步长得到近似  $\pi$ 
        printf("pi is approximately: %f\n", pi);
    }

    // 6. 结束 MPI 环境
    MPI_Finalize();

    return 0;
}

```

3.5 MapReduce 编程

3.5.1 基本语法

MapReduce 是一种面向大规模数据处理的并行编程模型，其核心思想是将计算分解为两步: Map (映射) 和 Reduce (归约)。

Map 函数负责处理输入数据，将原始数据集的每一条记录映射成一个中间的键值对 (key, value)。在 Map 函数中，通常只处理当前输入片段，不依赖其他数据，因此天然支持并行执行。

Reduce 函数则处理 Map 生成的中间结果。系统会先按键 (key) 对所有中间键值对进行分组 (shuffle)，然后将每组具有相同键的所有值传递给 Reduce 函数，Reduce 函数对这些值进行归约、汇总或其他计算操作，输出最终结果。

此外，MapReduce 通常提供一些基础类型和函数接口，例如输入和输出格式、键值类型以及可选的组合函数来在 Map 阶段进行局部归约，减少数据传输量。开发者只需定义 Map 和 Reduce 的逻辑，框架负责处理数据分片、任务调度、网络传输和容错机制。

一个 C++ 伪代码风格的简单示例如下：

```
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>
#include <mutex>
#include <thread>

// 示例输入数据：文本分片
std::vector<std::string> input_data = {
    "hello world hello",
    "world map reduce map",
    "hello map reduce"
};

// Map 阶段：将文本切分为单词并生成 (key, value) 对
void map_function(const std::vector<std::string>& data_chunk,
                  std::unordered_map<std::string, int>& local_map) {
    for (const auto& line : data_chunk) {
        size_t pos = 0, prev = 0;
        while ((pos = line.find(' ', prev)) != std::string::npos) {
            std::string word = line.substr(prev, pos - prev);
            local_map[word] += 1;
            prev = pos + 1;
        }
        std::string last_word = line.substr(prev);
        if (!last_word.empty()) local_map[last_word] += 1;
    }
}

// Combine 阶段：局部合并 Map 的结果，减少传输数据
void combine_function(const std::unordered_map<std::string, int>& local_map,
                     std::unordered_map<std::string, int>& combined_map,
```

```

        std::mutex& mtx) {
    std::lock_guard<std::mutex> lock(mtx); // 避免并发写冲突
    for (const auto& kv : local_map) {
        combined_map[kv.first] += kv.second;
    }
}

// Reduce 阶段：最终归约，计算全局单词计数
void reduce_function(const std::unordered_map<std::string, int>& combined_map,
                    std::unordered_map<std::string, int>& final_result) {
    for (const auto& kv : combined_map) {
        final_result[kv.first] += kv.second;
    }
}

int main() {
    const int num_threads = 3; // 使用线程模拟 MapReduce 的并行性
    std::vector<std::thread> threads;
    std::vector<std::unordered_map<std::string, int>> local_maps(num_threads);

    // 1. Map 阶段（并行执行）
    for (int t = 0; t < num_threads; ++t) {
        // 每个线程处理一个文本分片
        std::vector<std::string> chunk = { input_data[t] };
        threads.emplace_back(map_function, chunk, std::ref(local_maps[t]));
    }
    for (auto& th : threads) th.join();

    // 2. Combine 阶段（将局部 Map 合并到全局 combined_map）
    std::unordered_map<std::string, int> combined_map;
    std::mutex mtx;
    threads.clear();
    for (int t = 0; t < num_threads; ++t) {
        threads.emplace_back(combine_function, std::ref(local_maps[t]),
                             std::ref(combined_map), std::ref(mtx));
    }
    for (auto& th : threads) th.join();

    // 3. Reduce 阶段（全局归约，得到最终结果）
    std::unordered_map<std::string, int> final_result;
    reduce_function(combined_map, final_result);

    // 输出最终结果
    std::cout << "Word Count Results:\n";
    for (const auto& kv : final_result) {
        std::cout << kv.first << " : " << kv.second << "\n";
    }

    return 0;
}

```

- Map 阶段

- 每个线程独立处理一个文本分片，将每行拆成单词并计数。
- 输出的是局部 `unordered_map<string, int>`。
- **Combine 阶段**
 - 将每个线程的局部 Map 合并到一个全局 Map，减少在 Reduce 阶段的计算量。
 - 使用 `mutex` 防止并发写冲突。
- **Reduce 阶段**
 - 对 Combine 后的全局 Map 进行最终归约，得到全局单词计数结果。
- **并行执行**
 - 使用 `std::thread` 模拟 MapReduce 中的并行 Map 与 Combine。
 - Reduce 阶段是串行的 (单节点归约)，可以扩展为多线程归约或分布式 Reduce。

3.5.2 信息检索

信息检索算法的目标是：

给定一个大的文本集合，找到包含指定关键字的文档，或者统计每个单词出现在哪些文档中。

(Map 阶段)

输入: 单个文档 (`doc_id, text`)

处理逻辑:

- 将文本切分成单词。
- 对每个单词生成中间键值对 (`word, doc_id`)，也可以附加次数 (`word, (doc_id, count)`)

(Shuffle / Combine 阶段)

Map 阶段生成的 (`key, value`) 对会被框架按 key 进行分组，

即相同单词的所有中间结果会被发送到同一个 Reduce 任务。

Combine 是可选步骤，用于在 Map 端做局部聚合，减少数据传输量。

(Reduce 阶段)

输入: (`word, list_of_values`)，其中 `list_of_values` 是所有包含该单词的文档信息。

处理逻辑:

- 对相同单词的所有文档进行归并
- 统计每个单词在每个文档中的出现次数 (可选)
- 输出最终倒排索引 (`word, [(doc_id, count), ...]`)

C++ 风格的伪代码如下:

```
#include <string>
#include <vector>
#include <unordered_map>
#include <map>
#include <sstream>
#include <iostream>
```

```

using namespace std;

//----- Map 阶段 -----//
class Mapper {
public:
    // 输入: doc_id, 文本内容
    vector<pair<string, string>> map(const string& doc_id, const string& text) {
        vector<pair<string, string>> kv_pairs;
        istringstream iss(text);
        string word;
        while (iss >> word) {
            // 生成中间键值对 (word, doc_id)
            kv_pairs.emplace_back(word, doc_id);
        }
        return kv_pairs;
    }
};

//----- Combine 阶段 -----//
class Combiner {
public:
    // 将 Mapper 输出做本地聚合, 统计单词在单文档的次数
    vector<pair<string, pair<string, int>>> combine(const vector<pair<string, string>>&
kv_pairs) {
        unordered_map<string, unordered_map<string, int>> local_count;
        for (const auto& kv : kv_pairs) {
            const string& word = kv.first;
            const string& doc_id = kv.second;
            local_count[word][doc_id]++;
        }

        // 展平为 vector 便于 Shuffle 阶段传输
        vector<pair<string, pair<string, int>>> combined;
        for (auto& w : local_count) {
            for (auto& d : w.second) {
                combined.emplace_back(w.first, make_pair(d.first, d.second));
            }
        }
        return combined;
    }
};

//----- Reduce 阶段 -----//
class Reducer {
public:
    // 输入: key -> word, values -> list of (doc_id, count)
    unordered_map<string, vector<pair<string, int>>> reduce(
        const map<string, vector<pair<string, int>>>& grouped_data)
    {
        unordered_map<string, vector<pair<string, int>>> inverted_index;
        for (const auto& entry : grouped_data) {
            const string& word = entry.first;
            const vector<pair<string, int>>& values = entry.second;

```

```

        // 归并到最终倒排索引
        unordered_map<string, int> doc_count;
        for (auto& val : values) {
            doc_count[val.first] += val.second;
        }

        // 转换为 vector 存储
        for (auto& d : doc_count) {
            inverted_index[word].emplace_back(d.first, d.second);
        }
    }
    return inverted_index;
}

};

//----- 主流程 -----//
int main() {
    // 示例文档集合
    vector<pair<string, string>> documents = {
        {"doc1", "apple banana apple"},
        {"doc2", "banana orange apple"},
        {"doc3", "orange banana"}
    };

    Mapper mapper;
    Combiner combiner;
    Reducer reducer;

    // Map 阶段
    vector<pair<string, string>> all_kv;
    for (auto& doc : documents) {
        auto kv = mapper.map(doc.first, doc.second);
        all_kv.insert(all_kv.end(), kv.begin(), kv.end());
    }

    // Combine 阶段
    auto combined_kv = combiner.combine(all_kv);

    // Shuffle 阶段：按 word 分组
    map<string, vector<pair<string, int>>> grouped_data;
    for (auto& kv : combined_kv) {
        grouped_data[kv.first].push_back(kv.second);
    }

    // Reduce 阶段
    auto inverted_index = reducer.reduce(grouped_data);

    // 输出倒排索引
    for (auto& entry : inverted_index) {
        cout << entry.first << " -> ";
        for (auto& d : entry.second) {
            cout << "(" << d.first << ", " << d.second << ") ";
        }
    }
}

```

```

    }
    cout << endl;
}

return 0;
}

```

3.5.3 图算法

MapReduce 可以处理图算法的主要思路是将图数据 (节点和边) 表示成键值对, 然后通过多轮 MapReduce 迭代计算图上的指标, 如最短路径、PageRank、连通分量等. 假设图用邻接表表示, 每个节点包含:

- 节点 ID
- 当前已知的距离 (最短路径估计)
- 邻居列表 (可带边权)

(Map 阶段)

对每个节点, 将当前节点的距离信息传播给所有邻居.

输出键值对 `(neighbor_ID, new_distance)`, 表示如果通过当前节点到邻居的距离.

同时保留原节点信息, 用于 Reduce 阶段更新节点状态.

(Shuffle / Combine 阶段)

Map 产生的 `(key, value)` 对按 key (节点 ID) 分组,

确保同一个节点的所有距离信息被送到同一个 Reduce 任务.

(Reduce 阶段)

对每个节点, 取所有传来的距离的最小值, 更新该节点的最短路径估计.

将更新后的节点信息作为下一轮迭代的输入.

重复上述过程, 直到没有节点的最短路径发生变化.

这种迭代模式类似于 **Bellman-Ford 算法**, 每轮 MapReduce 相当于一次松弛操作.

C++ 风格的伪代码如下:

```

#include <string>
#include <vector>
#include <unordered_map>
#include <iostream>
#include <limits>
using namespace std;

const int INF = numeric_limits<int>::max();

// 节点结构
struct Node {
    string id; // 节点ID
    int distance; // 当前最短路径估计
    vector<pair<string, int>> edges; // 邻居列表, pair<邻居ID, 权重>
};

```



```

//----- Map 阶段 -----//
vector<pair<string, int>> map(const Node& node) {
    vector<pair<string, int>> output;

    // 将当前节点距离传播给邻居
    if (node.distance < INF) {
        for (auto& edge : node.edges) {
            string neighbor_id = edge.first;
            int weight = edge.second;
            int new_dist = node.distance + weight;
            output.emplace_back(neighbor_id, new_dist);
        }
    }

    // 输出自身节点信息, 确保 Reduce 可以更新节点邻居列表
    // key = node.id, value = -1 表示保留原节点结构
    output.emplace_back(node.id, -1);
    return output;
}

//----- Reduce 阶段 -----//
Node reduce(const string& node_id, const vector<pair<string, int>>& values, const Node&
old_node) {
    int min_dist = old_node.distance;

    // 遍历 Map 传来的距离信息
    for (auto& val : values) {
        int dist = val.second;
        if (dist >= 0 && dist < min_dist) {
            min_dist = dist;
        }
    }

    // 更新节点信息
    Node new_node = old_node;
    new_node.distance = min_dist;
    return new_node;
}

//----- 主流程 -----//
int main() {
    // 示例图: 节点ID -> Node
    unordered_map<string, Node> graph;

    // 初始化节点
    graph["A"] = {"A", 0, {{ "B", 1}, {"C", 4}}};
    graph["B"] = {"B", INF, {{ "C", 2}, {"D", 5}}};
    graph["C"] = {"C", INF, {{ "D", 1}}};
    graph["D"] = {"D", INF, {}};

    bool changed = true;
    int iteration = 0;

```

```

while (changed) {
    changed = false;
    iteration++;
    cout << "Iteration " << iteration << endl;

    // Map 阶段
    unordered_map<string, vector<pair<string, int>>> mapped;
    for (auto& kv : graph) {
        auto node = kv.second;
        auto outputs = map(node);
        for (auto& out : outputs) {
            mapped[out.first].push_back(out);
        }
    }

    // Reduce 阶段
    for (auto& kv : mapped) {
        string node_id = kv.first;
        vector<pair<string, int>>& values = kv.second;
        Node old_node = graph[node_id];
        Node new_node = reduce(node_id, values, old_node);
        if (new_node.distance != old_node.distance) {
            changed = true;
            graph[node_id] = new_node;
        }
    }
}

// 输出结果
cout << "Shortest distances:" << endl;
for (auto& kv : graph) {
    cout << kv.first << ": " << kv.second.distance << endl;
}

return 0;
}

```

The End

